

Superscalar Architecture

Mentorship Project Guidelines

Comprehensive Theory: Datapaths, Pipelining & Hazards

Technical Mentor

IIT Bombay

Summer 2026

Contents

1	Week 0 — Optional Catch-up (for 1st-year enthusiasts)	3
2	Week 1 — Datapaths, Pipelining & Hazards	4
3	Comprehensive Theory 1: The Single-Cycle Datapath	5
3.1	State Elements and Combinational Logic	5
3.2	The Anatomy of Execution: Tracing the 1 st Instruction	5
3.3	The Critical Path & Clock Frequency Limit	7
3.4	The Fatal Flaws of the Single-Cycle Design	8
3.4.1	1. Clock Cycle Waste on Shorter Instructions	8
3.4.2	2. Massive Hardware Duplication (Structural Constraints)	8
4	Comprehensive Theory 2: The Multi-Cycle Datapath	10
4.1	Hardware Sharing and Intermediate Registers	10
4.2	The 5-Step FSM Breakdown	10
4.3	Performance Dynamics of Multi-Cycle	12
5	Comprehensive Theory 3: The Need for Pipelining	14
5.1	The Dichotomy of the Pre-Pipelined Era	14
5.2	The Pipelining Breakthrough	15
6	Comprehensive Theory 4: The Pipelined Datapath	16
6.1	The Role of Pipeline Registers (Buffers)	16
6.2	Step-by-Step Anatomy of Pipelined Execution	16
6.3	The Cycle-by-Cycle Overlap (Space-Time)	18
7	Comprehensive Theory 5: Structural Hazards	20
8	Comprehensive Theory 6: Data Hazards & Forwarding	20
8.1	Read-After-Compute (RAW)	20
8.2	Read-After-Load	21
9	Comprehensive Theory 7: Control Hazards	21
9.1	Resolving Control Hazards	22
10	Comprehensive Theory 8: Beyond Scalar — The Need for Superscalar	22

1 Week 0 — Optional Catch-up (for 1st-year enthusiasts)

Goal

Ensure every mentee can write and simulate a basic Verilog module before Week 1.

Topics

- Combinational vs sequential logic; blocking vs non-blocking assignments.
- 5-stage MIPS pipeline — structure, not yet hazards.
- Quartus Prime Lite + ModelSim-Intel Starter: install, create project, simulate a 4-bit counter.

Resources

- Chapter 4 of Patterson & Hennessy — Computer Organization and Design (the green book). [*Click here to open/download Book PDF*]
- Intel FPGA University Program tutorials for Quartus + ModelSim setup. [*Click here to open/download Quartus Guidelines PDF*]
- NPTEL — Computer Architecture and Organization by Prof. Smruti Sarangi (select lectures).

Deliverable

- Screenshot of a working 4-bit counter waveform in ModelSim, plus Quartus compile report.

2 Week 1 — Datapaths, Pipelining & Hazards

Goal

Understand the evolution of processor datapaths (Single-Cycle and Multi-Cycle) in rigorous detail, and become fluent in pipelining, hazard detection, and forwarding — the absolute foundation for everything we will cover regarding superscalar architectures.

Mentor Session (2 hours)

- Deep architectural evolution from Single-Cycle to Multi-Cycle datapaths.
- The mathematical and architectural need for pipelining.
- Quick recap of the 5-stage pipeline; CPI, throughput, and speedup math.
- Structural, data (RAW, WAR, WAW), and control hazards with concrete MIPS assembly examples.
- Forwarding paths, pipeline stalls, delayed branches vs. branch prediction.
- Why scalar pipelines cap at $IPC = 1$ — motivating the need for superscalar designs.

Mentee Work (6–8 hours)

- Read: Patterson & Hennessy Ch. 4.1–4.9.
- Hand-trace 3 assembly sequences showing each hazard type; identify stalls and forwarding paths.
- Complete the short quiz (mentor-set) on hazard detection provided at the end of each theory section.

Deliverable

- PDF containing hand-traces and quiz answers, uploaded and shared on your project GitHub repository.

3 Comprehensive Theory 1: The Single-Cycle Datapath

Before we can appreciate the nuances of pipelining and superscalar execution, we must exhaustively analyze the foundational processor design: the Single-Cycle Datapath. The single-cycle datapath is the most straightforward conceptual implementation of an instruction set architecture (ISA). As the name implies, in a single-cycle implementation, every single instruction takes exactly one clock cycle to execute in its entirety. From the moment the instruction is fetched from memory to the moment its final results are permanently written to the register file or memory, the clock does not tick.

3.1 State Elements and Combinational Logic

To construct a datapath, we utilize two fundamentally different types of logic blocks:

1. **State Elements (Sequential Logic):** Elements like the Program Counter (PC), Register File, and Data Memory. These elements possess internal storage. Their outputs only change when they receive a clock edge (typically a positive rising edge). State elements dictate the boundaries of a clock cycle.
2. **Combinational Logic:** Elements like the Arithmetic Logic Unit (ALU), multiplexers, and adders. These have no internal state or memory. Their outputs are strictly a function of their current inputs. Once inputs are applied, the electrical signals propagate through the logic gates, and after a brief propagation delay, the output settles to its final value.

In a single-cycle datapath, the execution of an instruction is entirely combinational. The PC provides an address at the beginning of the clock cycle, and the signal naturally flows through all the combinational logic and intermediate state elements (operating in read mode) until it reaches the input of the final state element, where it waits for the next clock edge to be permanently written.

3.2 The Anatomy of Execution: Tracing the `lw` Instruction

To thoroughly understand the single-cycle datapath, we trace the execution of the most complex instruction in the standard MIPS instruction set: the Load Word (`lw`) instruction. The `lw` instruction is the "worst-case scenario" because it requires the processor to utilize almost every major hardware component sequentially.

Let's break down the execution path of the instruction `lw $rt, offset($rs)` step-by-step:

Step 1: Instruction Fetch (IF)

The execution begins at the rising edge of the clock. The Program Counter (PC) outputs the 32-bit address of the current instruction.

- This address is routed to the **Instruction Memory**. Because reading from memory acts like combinational logic in this basic model, the memory outputs the 32-bit instruction word after a specific delay.
- Concurrently, the PC address is routed to a dedicated **Adder** that calculates $PC+4$. This value will be routed back to the PC to prepare for the next instruction, waiting at the input of the PC for the next clock edge.

Step 2: Instruction Decode & Register Read (ID)

The 32-bit instruction word is now available and is split into its respective fields:

- Bits [31:26] (the opcode) are routed to the **Control Unit**, which begins deciphering the instruction and turning on the correct control signals (**MemRead**, **ALUSrc**, **RegWrite**, etc.).
- Bits [25:21] (**rs**) and Bits [20:16] (**rt**) are routed to the **Register File** read ports. The Register File immediately outputs the values stored in these registers. For **lw**, we primarily care about the base address in **rs**.
- Bits [15:0] (the immediate offset) are routed to the **Sign-Extension Unit**, which expands the 16-bit value into a 32-bit value, preserving the sign, preparing it for ALU addition.

Step 3: Execution / Address Calculation (EX)

The signals have now propagated through the Register File and Sign-Extension Unit.

- The value read from the base register (**rs**) arrives at the first input of the **ALU**.
- The 32-bit sign-extended offset arrives at a multiplexer. The Control Unit asserts the **ALUSrc** signal to 1, routing the immediate value to the second input of the ALU.
- The Control Unit commands the ALU to perform an **Addition**. After the ALU's propagation delay, it outputs the effective memory address.

Step 4: Memory Access (MEM)

The effective memory address leaves the ALU and is routed directly into the address port of the **Data Memory**.

- The Control Unit asserts the **MemRead** signal.
- The Data Memory accesses the requested address and, after its inherent read delay, outputs the 32-bit data onto its read data port.

Step 5: Write Back (WB)

The data fetched from the Data Memory travels through a multiplexer controlled by the `MemtoReg` signal (which is set to 1 for loads).

- The data arrives at the `Write Data` port of the Register File.
- Bits [20:16] (`rt`) are routed through a multiplexer (controlled by `RegDst = 0`) into the `Write Register` port.
- The Control Unit asserts the `RegWrite` signal to 1.
- The data simply waits at the input port. When the *next* rising edge of the clock arrives, the data is permanently captured inside the register file, and the cycle completes.

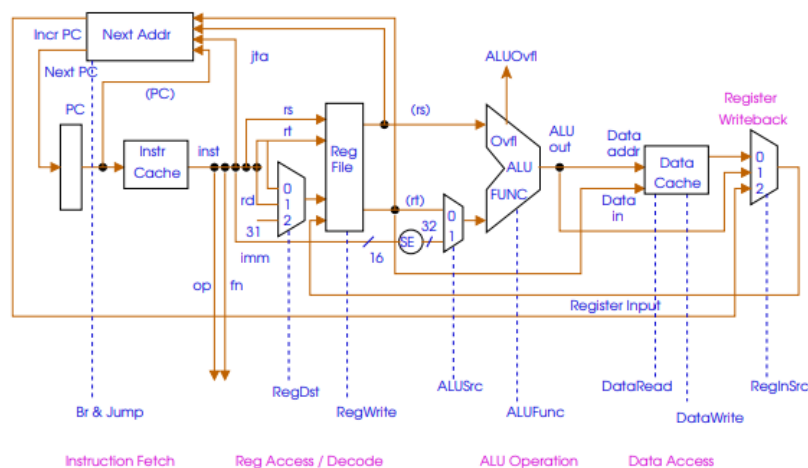


Figure 1: Single-cycle datapath

3.3 The Critical Path & Clock Frequency Limit

In a single-cycle design, the clock cycle time is strictly bounded by the **critical path**—the absolute longest possible path an electrical signal must travel through the hardware from the start of the cycle to the end. For the MIPS architecture, the `lw` instruction represents this critical path because it utilizes the Instruction Memory, Register File, ALU, Data Memory, and Register File (write) in a single continuous chain.

Let's assume hypothetical, yet realistic, propagation delays for our hardware components:

- Instruction/Data Memory access: 200ps
- ALU / Adders: 100ps

- Register File Read/Write: 50ps

The critical path for the `lw` instruction would be calculated as:

$$T_{cycle} = T_{IMem} + T_{RegRead} + T_{ALU} + T_{DMem} + T_{RegWrite}$$

$$T_{cycle} = 200ps + 50ps + 100ps + 200ps + 50ps = 600ps$$

Therefore, the minimum clock cycle time is 600 picoseconds. The maximum clock frequency of this processor is the inverse of the period:

$$F_{max} = \frac{1}{T_{cycle}} = \frac{1}{600 \times 10^{-12} \text{ s}} \approx 1.66 \text{ GHz}$$

3.4 The Fatal Flaws of the Single-Cycle Design

While conceptually elegant and extremely easy to implement and verify, the single-cycle datapath is practically extinct in modern high-performance computing due to two severe inefficiencies:

3.4.1 1. Clock Cycle Waste on Shorter Instructions

The clock cycle must be long enough to safely accommodate the *slowest* instruction (`lw`). However, what happens during an `add` instruction? An `add` instruction only requires the Instruction Memory (200ps), Register Read (50ps), ALU (100ps), and Register Write (50ps). It skips Data Memory entirely.

$$T_{add_required} = 200ps + 50ps + 100ps + 50ps = 400ps$$

In a single-cycle design, the `add` instruction finishes its work in 400ps, but the hardware simply sits completely idle for the remaining 200ps waiting for the clock to tick. If a program consists mostly of arithmetic instructions, nearly 33% of the processor's active time is wasted doing absolutely nothing.

3.4.2 2. Massive Hardware Duplication (Structural Constraints)

Because the clock does not tick during an instruction's execution, a hardware resource can only be used exactly *once* per clock cycle. We cannot time-multiplex hardware.

- We **must** have separate Instruction and Data memories because we cannot read an instruction and read data from the same physical memory module in the same cycle.
- We **must** have separate dedicated adders to compute $PC + 4$ and branch target addresses, because the main ALU is fully occupied performing the primary mathematical operation or calculating memory addresses.

This results in a massively bloated silicon footprint, leading to high cost and high power consumption for a severely limited clock speed.

Concept Check 1

1. If the latency of the Data Memory increases by 50ps to 250ps, how does this specifically affect the execution time of an **add** instruction in a single-cycle datapath?
2. Explain the state of the **MemtoReg** control signal during an **add** instruction and a **beq** (branch equal) instruction.
3. Why must the single-cycle datapath have separate adders for the PC, rather than using the main ALU?

4 Comprehensive Theory 2: The Multi-Cycle Datapath

To resolve the severe inefficiencies of the single-cycle design—specifically the wasted time and the massive hardware duplication—computer architects developed the **Multi-Cycle Datapath**.

The core philosophy of the multi-cycle datapath is: *Break the execution of an instruction into multiple, smaller, discrete steps, where each step takes exactly one short clock cycle.*

4.1 Hardware Sharing and Intermediate Registers

By breaking the instruction into steps bounded by clock cycles, we can reuse the same functional units across different clock cycles within the execution of a single instruction. This time-multiplexing solves the hardware duplication flaw:

- We **no longer need** separate Instruction and Data memories. A single, unified Memory module can be used to fetch the instruction in Step 1, and read/write data in Step 4.
- We **no longer need** extra adders for the PC. The main ALU can be used to increment the PC in Step 1, compute branch targets in Step 2, and perform instruction-specific arithmetic in Step 3.

However, because multiple clock cycles are passing, the output of a hardware unit from Step 1 will disappear in Step 2 unless we explicitly save it. Therefore, the multi-cycle datapath introduces several **intermediate registers** (architecturally invisible to the programmer) to hold state between cycles:

- **Instruction Register (IR):** Holds the 32-bit instruction fetched from memory in Cycle 1 so it can be decoded in subsequent cycles.
- **Memory Data Register (MDR):** Holds data fetched from memory during a load operation.
- **A & B Registers:** Hold the source values read from the Register File.
- **ALUOut:** Holds the mathematical output of the ALU, waiting to be written to memory or the register file.

4.2 The 5-Step FSM Breakdown

Execution in a multi-cycle datapath is managed by a complex Control Finite State Machine (FSM). Let us trace the absolute maximum number of steps (five) that an instruction can take.

Cycle 1: Instruction Fetch

In the first clock cycle, the processor does the same thing regardless of the instruction type.

- The memory is addressed using the PC. The read data is placed into the **Instruction Register (IR)**.
- Simultaneously, the main ALU adds 4 to the PC. The result is routed back and written into the PC.

Hardware Used: Memory, ALU, PC.

Cycle 2: Instruction Decode & Register Fetch

Because the instruction format is highly regular, we can read the registers before we even fully understand what the instruction is.

- Registers **rs** and **rt** are read from the Register File into registers **A** and **B**.
- The ALU **speculatively** calculates the branch target address: it takes the PC and adds it to the sign-extended, shifted-by-2 immediate field. The result is stored in **ALUOut**. If the instruction turns out not to be a branch, this calculated value is simply ignored.

Hardware Used: Register File, Sign-Extension Unit, ALU.

Cycle 3: Execution, Memory Address Computation, or Branch Completion

This is the cycle where instructions branch into different states based on their opcode.

- **Memory Reference (lw/sw):** The ALU adds register **A** to the sign-extended immediate value to compute the effective address, storing it in **ALUOut**.
- **Arithmetic (R-type):** The ALU performs the specified operation (e.g., ADD, SUB, AND) on registers **A** and **B**, storing the result in **ALUOut**.
- **Branch (beq):** The ALU subtracts **B** from **A**. If the zero flag is set, the PC is updated with the branch target address calculated in Cycle 2. *The branch instruction finishes execution here.*

Hardware Used: ALU.

Cycle 4: Memory Access or R-type Completion

- **Memory Reference (lw):** The memory is read using the address in **ALUOut**. The data is placed in the **Memory Data Register (MDR)**.

- **Memory Reference (sw):** The data in register **B** is written to memory at the address in **ALUOut**. *The store instruction finishes execution here.*
- **Arithmetic (R-type):** The data in **ALUOut** is written to the destination register (**rd**) in the Register File. *The R-type instruction finishes execution here.*

Hardware Used: Memory or Register File.

Cycle 5: Memory Read Completion

- **Load Word (lw) Only:** The data in the **MDR** is written back to the destination register (**rt**) in the Register File. *The load instruction finishes execution here.*

Hardware Used: Register File.

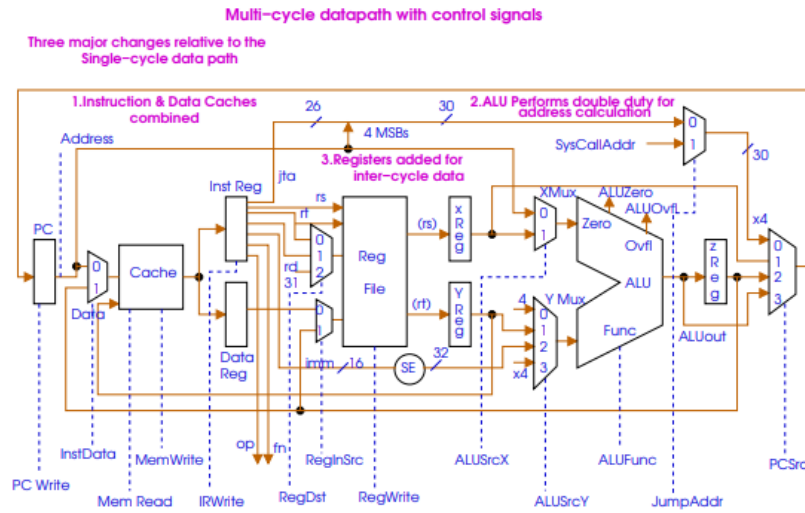


Figure 2: Multi-cycle datapath

4.3 Performance Dynamics of Multi-Cycle

The clock cycle time in a multi-cycle datapath is determined not by the entire **lw** instruction, but by the *slowest individual step* (e.g., a single memory access taking 200ps). This allows the clock frequency to jump from 1.66 GHz in the single-cycle design to a staggering 5.0 GHz.

Furthermore, instructions no longer waste cycles. A **beq** instruction finishes and yields control in 3 cycles. An **add** finishes in 4 cycles. An **lw** takes the full 5.

However, we face a mathematical reality check. While the clock frequency is much higher, the CPI (Cycles Per Instruction) has also risen from exactly 1 to an average of roughly 4.0. The total latency of a single instruction remains fundamentally unchanged. More importantly, the multi-cycle processor still only processes **one instruction at a time**.

Concept Check 2

1. In a multi-cycle datapath, why is the Instruction Register (IR) necessary, while it was not needed in the single-cycle datapath?
2. How does the multi-cycle datapath handle hardware utilization during a branch instruction differently than a single-cycle datapath?
3. If an **add** instruction takes 4 cycles to complete in a 5 GHz multi-cycle processor, what is its total latency in picoseconds?

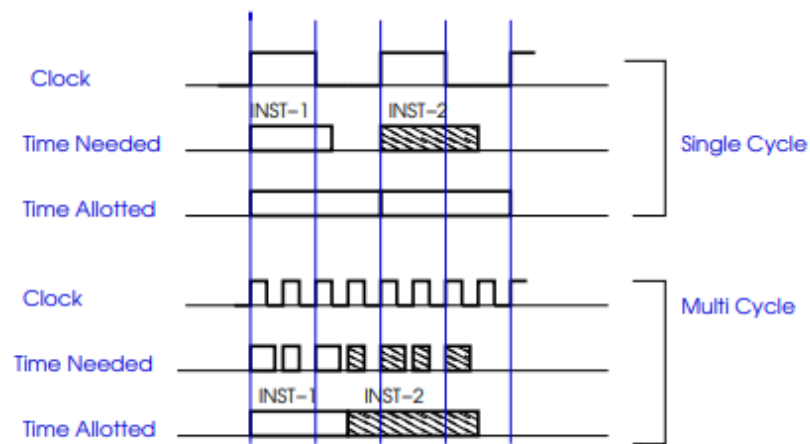


Figure 3: Single-cycle or Multi-cycle Execution Comparison

5 Comprehensive Theory 3: The Need for Pipelining

To truly understand why pipelining was invented, we must look at the **Iron Law of Processor Performance**. The CPU execution time for a given program is dictated by the following equation:

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

To make a computer faster, an architect must minimize this equation.

- **Instruction Count:** Largely determined by the ISA and the Compiler. Hardware designers have little direct control over this once the ISA is fixed.
- **Clock Cycle Time (T_{cycle}):** We want to make the clock as fast as possible.
- **Cycles Per Instruction (CPI):** We want this to be as close to 1.0 (or lower) as possible.

5.1 The Dichotomy of the Pre-Pipelined Era

Let's mathematically compare our two previous datapaths executing a program of 1,000 instructions. We will use the latencies defined earlier: Memory = 200ps, ALU = 100ps, Register = 50ps.

1. Single-Cycle Processor:

- T_{cycle} must cover the longest path (600ps).
- CPI is exactly 1 for all instructions.
- Total Time = $1000 \times 1 \times 600ps = 600,000ps$

2. Multi-Cycle Processor:

- T_{cycle} covers the longest individual step (Memory = 200ps).
- CPI varies (e.g., 3 for branch, 4 for ALU, 5 for load). Assume an average CPI of 4.0.
- Total Time = $1000 \times 4.0 \times 200ps = 800,000ps$

Wait! The multi-cycle processor is actually slower! Why? Because we introduced overhead. We broke the 100ps ALU operation into a 200ps clock cycle step to standardize the clock.

We face an architectural wall:

- If we want **CPI = 1**, we must use the single-cycle design, which results in a **horribly slow clock**.
- If we want a **fast clock**, we must use the multi-cycle design, which results in a **horribly high CPI**.

5.2 The Pipelining Breakthrough

Pipelining is the architectural breakthrough that shatters this dichotomy. It attempts to give us the best of both worlds: a CPI approaching 1.0, while maintaining a fast clock frequency identical to the multi-cycle datapath.

Pipelining achieves this not by making individual instructions faster, but by aggressively improving the **throughput** of the entire workload. If we have a massive sequence of instructions, we do not wait for Instruction 1 to finish completely before fetching Instruction 2. We overlap their execution perfectly in time.

6 Comprehensive Theory 4: The Pipelined Datapath

The pipelined datapath marries the single-cycle datapath's structural simplicity with the multi-cycle datapath's high clock speed. It takes the five distinct steps of execution (Fetch, Decode, Execute, Memory, Write Back) and turns them into an assembly line.

6.1 The Role of Pipeline Registers (Buffers)

If five instructions are moving through the hardware at the exact same time, we must physically divide the datapath to prevent signals from different instructions from colliding. We achieve this by inserting massive **Pipeline Registers** (state elements) between each stage.

1. **IF/ID Register:** Placed between Instruction Fetch and Instruction Decode. It holds the 32-bit instruction fetched from memory and the $PC + 4$ value.
2. **ID/EX Register:** Placed between Decode and Execute. It holds the values read from the Register File, the sign-extended immediate, $PC + 4$ (for branch targets), the target register addresses, and all control signals required for the EX, MEM, and WB stages.
3. **EX/MEM Register:** Placed between Execute and Memory. It holds the ALU result (which could be a memory address or arithmetic result), the data to be stored to memory (for **sw**), the destination register address, and control signals for the MEM and WB stages.
4. **MEM/WB Register:** Placed between Memory and Write Back. It holds the data read from memory, the ALU result, the destination register address, and control signals for the WB stage.

These registers are clocked. On every rising edge of the clock, the data calculated by Stage N is captured by the Pipeline Register and handed to Stage $N + 1$.

6.2 Step-by-Step Anatomy of Pipelined Execution

Let's trace a `lw $s1, 100($s2)` instruction flowing step-by-step through the pipeline:

Cycle 1: IF Stage

- The PC addresses the Instruction Memory.
- The memory outputs the 32-bit instruction word for `lw`.
- The PC adder calculates $PC + 4$.

- **End of Cycle 1:** The instruction word and $PC + 4$ are written into the **IF/ID Pipeline Register**. The PC itself is updated with $PC + 4$.

Cycle 2: ID Stage

- The `lw` instruction is now safely held in the IF/ID register. (Meanwhile, the IF hardware is fetching the *next* instruction).
- The Control Unit reads the opcode and generates control signals. Crucially, these control signals don't act immediately; they are bundled and passed down the pipeline alongside the data.
- The Register File reads `$s2`. The immediate field 100 is sign-extended.
- **End of Cycle 2:** The value of `$s2`, the sign-extended 100, the destination register `$s1`, and the bundled Control Signals are written into the **ID/EX Pipeline Register**.

Cycle 3: EX Stage

- The ID/EX register provides the inputs.
- The ALU takes the value of `$s2` and adds it to 100 to calculate the effective memory address.
- **End of Cycle 3:** The newly calculated address, the destination register `$s1`, and the remaining MEM/WB Control Signals are written into the **EX/MEM Pipeline Register**.

Cycle 4: MEM Stage

- The EX/MEM register provides the memory address.
- The Control Unit signals (passed via the pipeline register) assert `MemRead`. The Data memory accesses the address and outputs the data.
- **End of Cycle 4:** The fetched data, the destination register `$s1`, and the WB Control Signals are written into the **MEM/WB Pipeline Register**.

Cycle 5: WB Stage

- The MEM/WB register provides the data and the destination register `$s1`.
- The data is routed back to the Register File. The `RegWrite` control signal allows the Register File to update `$s1`.
- **End of Cycle 5:** The instruction is officially completed and exits the pipeline.

6.3 The Cycle-by-Cycle Overlap (Space-Time)

To see the true power of the pipeline, we map 5 arbitrary sequential instructions on a Space-Time diagram:

- **Cycle 1:** Inst 1 is in IF.
- **Cycle 2:** Inst 1 moves to ID. Inst 2 enters IF.
- **Cycle 3:** Inst 1 moves to EX. Inst 2 moves to ID. Inst 3 enters IF.
- **Cycle 4:** Inst 1 moves to MEM. Inst 2 moves to EX. Inst 3 moves to ID. Inst 4 enters IF.
- **Cycle 5:** Inst 1 moves to WB. Inst 2 moves to MEM. Inst 3 moves to EX. Inst 4 moves to ID. Inst 5 enters IF. (*The pipeline is now full!*)
- **Cycle 6:** Inst 1 finishes. Inst 2 moves to WB. Inst 3 moves to MEM. Inst 4 moves to EX. Inst 5 moves to ID.

From Cycle 5 onward, **one instruction completes every single clock cycle**. Even though Inst 1 still took 5 clock cycles (its latency did not decrease), the throughput of the processor has reached 1 instruction per cycle, matching the Single-Cycle datapath but running at the massive 5.0 GHz clock speed of the Multi-Cycle datapath.

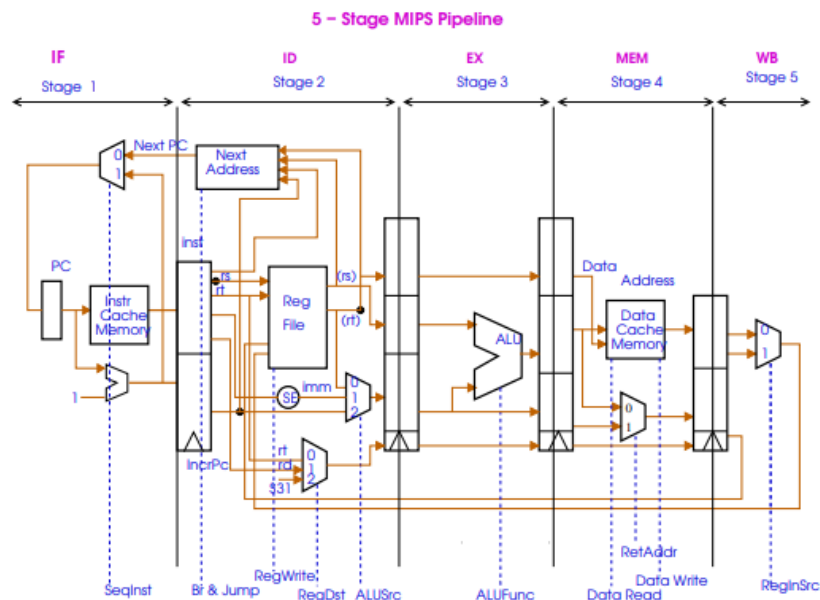


Figure 4: Pipelined Datapath with Pipeline Registers

Concept Check 3

1. At the end of Cycle 3, exactly what data is being held inside the **EX/MEM** pipeline register for an **add** instruction?
2. Why must the control signals generated in the ID stage be passed along the pipeline registers rather than being sent directly to the ALU, Data Memory, and Register File?
3. If a pipeline has 5 stages, and we execute 100 instructions, how many clock cycles will it take to finish in an ideal scenario (no stalls)?

7 Comprehensive Theory 5: Structural Hazards

While pipelining is elegant in theory, instructions overlapping in time create massive conflicts known as **Hazards**. A hazard prevents the next instruction from executing during its designated clock cycle.

A **Structural Hazard** arises when the physical hardware cannot support the combination of instructions that need to be executed in the exact same clock cycle.

The MIPS instruction set was explicitly designed to be pipelined, making it inherently easier to design without structural hazards. For example, if a pipeline utilized a single unified memory module for both instructions and data, reading data (in the MEM stage) and reading an instruction (in the IF stage) in the same cycle would cause a structural hazard. Two instructions cannot access the same single memory port simultaneously. MIPS resolves this conflict by implementing the **Harvard Architecture**: two separate caches, one dedicated instruction cache and one dedicated data cache.

Similarly, in Cycle 5 of the pipeline, an instruction in the WB stage is writing to the Register File. In that exact same cycle, an instruction in the ID stage is reading from the Register File. To prevent a structural hazard, the Register File must support multiple independent ports (e.g., two read ports and one write port), and must be designed to write on the falling edge of the clock and read on the rising edge, effectively splitting the cycle in half.

Concept Check 4

1. Propose a hardware modification to resolve a structural hazard caused by an ALU that is required to both compute an arithmetic result for the EX stage and increment the PC for the IF stage in the same cycle.

8 Comprehensive Theory 6: Data Hazards & Forwarding

Data hazards arise from the data dependence of one instruction on an earlier instruction that is still executing within the pipeline. Fundamentally, instances of data flowing backwards in time are hazards.

8.1 Read-After-Compute (RAW)

A Read-After-Compute hazard occurs when an arithmetic instruction relies on the result of an immediately preceding arithmetic instruction.

```
add $s0, $t0, $t1  # Does not write $s0 until stage 5 (WB)
sub $t2, $s0, $t3  # Requires $s0 in stage 2 (ID)
```

Because `add` does not write to `$s0` until its WB stage, but `sub` needs it in its ID stage, the data is missing. The `sub` instruction would read stale, incorrect data from the Register File.

Solution - Data Forwarding (Bypassing): Extra hardware is added to retrieve the missing item early from internal pipeline registers. The sum calculated by the `add` instruction actually exists at the end of its EX stage (Cycle 3). We do not have to wait for it to be written to the Register File in Cycle 5. As soon as the ALU creates the sum, it is held in the EX/MEM register. We add a **Forwarding Unit** and multiplexers to route that value directly from the EX/MEM register back to the input of the ALU for the `sub` instruction executing its EX stage in Cycle 4.

8.2 Read-After-Load

Data forwarding alone cannot prevent *all* pipeline data hazards. Consider a load instruction followed by a dependent instruction:

```
lw  $s0, 8($s1)    # Reads data from memory in stage 4 (MEM)
sub $t2, $s0, $t3   # Requires $s0 in stage 3 (EX)
```

The desired data from the `lw` instruction will not physically exist until it is read from the Data Memory at the end of stage 4 (MEM). However, the `sub` instruction needs that data at the beginning of its EX stage (stage 3). This is physically impossible; we cannot forward data backwards in time.

Solution - Pipeline Stalls & Reordering: This scenario requires a **Pipeline Stall** (inserting a 'bubble' or 'NOP'). A hardware **Hazard Detection Unit** detects the load-use condition. It halts the IF and ID stages (preventing the PC from incrementing) and forces the control signals for the EX stage to 0 (creating a bubble). This delays the `sub` instruction by one cycle, allowing the `lw` to reach the MEM stage and forward the data.

Alternatively, an optimizing compiler can reorder instructions to place an unrelated, independent instruction in the "load delay slot" directly after the load, avoiding the hardware stall entirely.

Concept Check 5

1. Hand-trace the `lw` followed by `sub` sequence. Draw the pipeline stages and indicate exactly where the bubble must be inserted.
2. Explain why Write-After-Write (WAW) hazards do not typically manifest in a strict, single-issue, 5-stage in-order MIPS pipeline.

9 Comprehensive Theory 7: Control Hazards

Control hazards occur when a pipeline decision must be based on the results of one instruction while other instructions are already executing. In a conditional branch instruction (`beq`), the target address and the branch outcome (taken or not taken) are

historically not known until the EX stage (Cycle 3). By that time, two sequential instructions have already entered the IF and ID stages. If the branch is taken, those two instructions were fetched by mistake and must be thrown away.

9.1 Resolving Control Hazards

1. **Extra Hardware (Early Evaluation):** Hardware can be modified to test registers, calculate the branch address, and update the PC during the second stage (ID) instead of EX. This reduces the branch penalty from two cycles to one cycle.
2. **Branch Prediction:** The processor predicts that the branch will *not* be taken (evaluates to false) and continues fetching sequential instructions. If the prediction is correct, the pipeline operates at full speed without penalty. If incorrect, the already started instructions must be stalled for a cycle and flushed.
3. **Static vs. Dynamic Prediction:** Static branch prediction is based on typical compiled code behavior (e.g., predicting that branches at the bottom of loops will usually be taken). Dynamic branch prediction relies on hardware: it looks up the address of the branch in a Branch Prediction Buffer to see if it was taken during its last execution, dynamically adjusting the fetch logic based on recent history.
4. **Delayed Branch:** A software approach that redefines the ISA architecture by instituting a "branch delay slot". The compiler is forced to place an instruction that *always* executes (regardless of the branch outcome) immediately after the branch instruction. The processor is designed to always execute the instruction following the branch before actually jumping. This eliminates the hazard entirely but restricts the compiler.

Concept Check 6

1. Contrast the performance penalty of a correct versus an incorrect dynamic branch prediction in a 5-stage pipeline.
2. Why is the delayed branch approach considered a "software" or compiler-driven solution rather than a purely hardware solution?

10 Comprehensive Theory 8: Beyond Scalar — The Need for Superscalar

Standard pipelined organizations, such as the 5-stage MIPS datapath, are fundamentally **scalar**. A scalar pipeline processes instructions sequentially through its stages. Because there is only one pipeline, it can fetch, decode, execute, and complete at most *one instruction per clock cycle*. Therefore, scalar pipelines are strictly capped at an Instructions Per Cycle (IPC) of 1 (or CPI of 1).

However, computer architects constantly strive for higher performance. Because single-cycle and multi-cycle datapaths hit severe latency limits, and single-issue pipelining cannot mathematically exceed an IPC of 1, the only way to achieve exponentially better performance is to issue multiple instructions simultaneously.

A **Super-scalar organization** features multiple, parallel, independent execution pipelines within the same processor core. By replicating execution units (e.g., having 3 ALUs, 2 Memory Access ports, and fetching 4 instructions at a time), the processor can fetch, decode, and execute several instructions at the exact same instant.

This architectural leap breaks the $\text{IPC} = 1$ barrier. However, it exponentially increases the complexity of resolving Structural, Data, and Control hazards, as the processor must now dynamically check for dependencies between multiple instructions executing simultaneously in the exact same pipeline stages. This transition into true parallel instruction execution forms the basis for the advanced Superscalar design projects we will tackle in the coming weeks.

Concept Check: Week 1 Synthesis

1. For a sequence of 3 instructions, calculate the total execution time for a Single-cycle (600ps total latency per instruction) versus a Pipelined architecture (assume 200ps clock cycle, ideal flow).
2. If a 5-stage scalar pipeline has an ideal IPC of 1, what primary architectural components must be replicated to upgrade it to a 2-way superscalar processor?
3. How do data hazards (RAW) become exponentially more complex to resolve in a superscalar architecture compared to a scalar pipeline?